

to “repeat” this initial allocation, R_N . We compute U_N by first rearranging Equation 3 to find the optimal compute budget for the unique tokens used (U_D). We input this value into the N_{opt} formula of Equation 3 to get $U_N = \min\{N_{opt}, N\}$. U_N thus corresponds to the compute-optimal number of parameters for U_D or less if $N < N_{opt}$. Once we have U_N , we compute the repeat value as $R_N = (N/U_N) - 1$.

To empirically explore the scaling behavior in a data-limited setting we train LLMs under these constraints. We consider three different experimental protocols in this work:

- *(Fixed Unique Data)* In §5 we fix the data constraint D_C and train models varying epochs and parameters. These experiments target *Allocation*, specifically tradeoff of D and N .
- *(Fixed FLOPs)* In §6 we fix the computation available and vary D_C (and thus also U_D and U_N). These experiments target *Return*, i.e. how well does repeating scale compared to having more unique data.
- *(Parametric Fit)* We fit a formula introduced in §3.1 on all our training runs and evaluate its predictive capability throughout §5 and §6.

Before discussing experimental results we describe the parametric assumptions.

3.1 Parametric Fit

To extrapolate scaling curves, it is necessary to incorporate repetition into the Chinchilla formula (Equation 2). We generalize Equation 2 by replacing D and N with terms corresponding to the *effective data* (D') and *effective model parameters* (N').

$$L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

Intuitively, D' should be smaller or equal to D where D is the total number of processed tokens since repeated tokens provide less useful information to the model than new ones. We use an *exponential decay* formulation, where the value of a data token processed loses roughly $(1 - 1/R_D^*)$ fraction of its value per repetition, where R_D^* is a learned constant. After some derivations and approximations (see Appendix A), this boils down to

$$D' = U_D + U_D R_D^* (1 - e^{-\frac{R_D}{R_D^*}}) \quad (5)$$

Note that for $R_D = 0$ (no repetitions), $D' = U_D = D$. For $R_D \ll R_D^*$, $e^{-R_D/R_D^*} \approx 1 - \frac{R_D}{R_D^*}$ and so

$$D' \approx U_D + U_D R_D^* (1 - 1 + R_D/R_D^*) = U_D (1 + R_D) = D$$

and hence in this case, repeated data is worth almost the same as fresh data. (This is also consistent with the predictions of the “deep bootstrap” framework [76].) As R_D grows, the value of repeated tokens tends to zero, and the effective data D' becomes much smaller than D . The formula implies that no matter how many times we repeat the data, we will not get a better loss than could be obtained with a single epoch on $U_D + U_D R_D^*$ fresh tokens.

Just as processing repeated tokens yields a diminishing return, both intuitively and empirically, models with sizes that vastly outstrip the available data also offer diminishing returns per parameter. Hence we use a symmetric formula for the number of effective parameters, where again R_N^* is learned,

$$N' = U_N + U_N R_N^* (1 - e^{-\frac{R_N}{R_N^*}}) \quad (6)$$

The learned constants R_D^* , R_N^* roughly correspond to the “half-life” of repeated data and excess parameters. For example, at $R_D = R_D^*$, the number of effective tokens D' is $U_D + U_D R_D^* (1 - e^{-1})$ which means that the $U_D R_D^*$ repeated tokens are worth on average $1 - 1/e$ fraction of fresh ones.

Using a methodology similar to [42], R_N^* and R_D^* can be fit on empirical measurements, which yields data-driven estimates. See Appendix A for more details on the derivations and the fitting procedure.

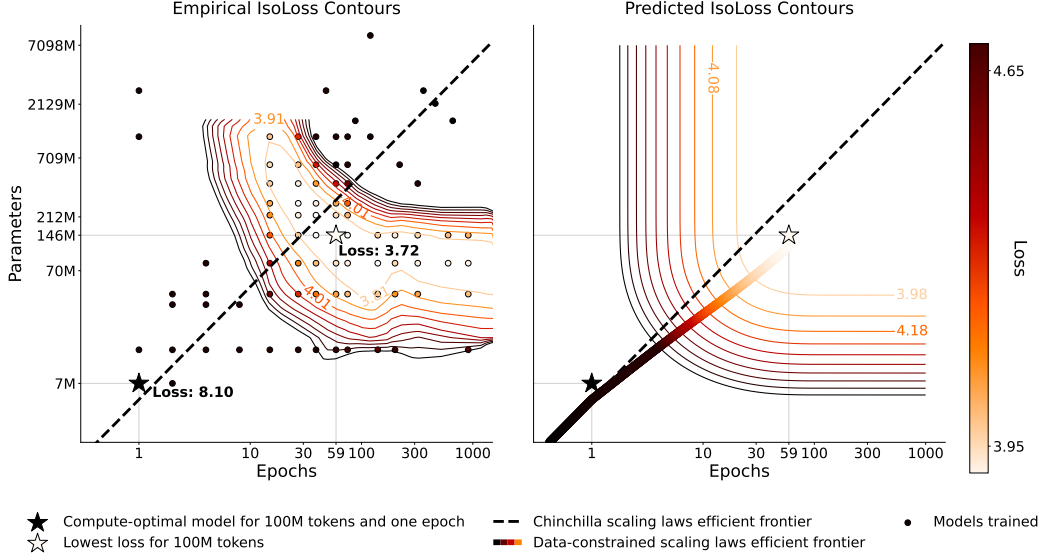


Figure 3: **IsoLoss contours for 100 million unique tokens.** (Left): 93 models trained with varying parameters and epochs on a fixed dataset. Contours show an interpolation of results with the same final test loss. (Right): Comparison with the loss predictions from our proposed scaling laws for the same budget of 100 million unique tokens and the predicted efficient frontier. The diminishing returns from training on repeated data can be seen in the increase in distance of the contour curves.

4 Experimental Setup

For all experiments, we train transformer language models with the GPT-2 architecture and tokenizer [88]. Models have up to 8.7 billion parameters and are trained for up to 900 billion total tokens. Following [42] we use cosine learning rate schedules that decay $10\times$ over the course of training for each model (different schedules led to different estimates in [46]). Unlike [46], we do not use early stopping to also explore the extent of overfitting when repeating. Other hyperparameters are based on prior work [89, 42] and detailed in Appendix S. Models are trained on subsets of C4 [90]. The data constraints are carefully defined to ensure maximal overlap as shown in Figure 2. Unlike [40], we always repeat the entire available data rather than subsets of it. Data is shuffled after each epoch. As repeating data can result in extreme overfitting (see Appendix H), we report loss on a held-out test set unless otherwise specified (see Appendix K). This contrasts training loss used in [42], but should not alter our findings as the held-out data stems from the same underlying dataset.

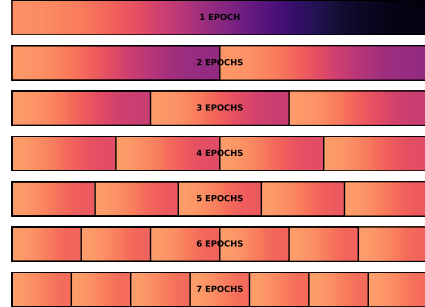


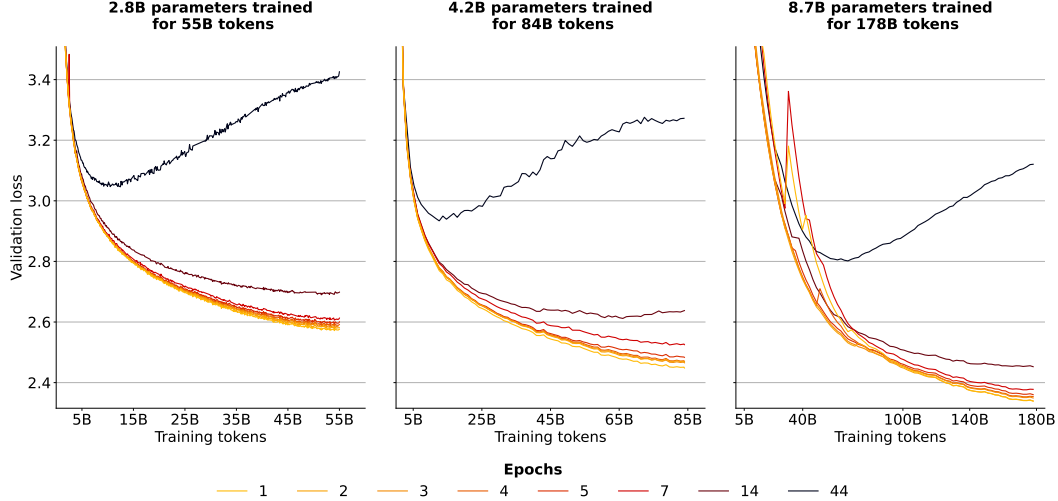
Figure 2: **Dataset setup.** We ensure that runs using less data (more epochs) always use a subset of the data used in runs with more data (fewer epochs).

5 Results: Resource Allocation for Data-Constrained Scaling

Our first experimental setting considers scaling in a setting where all models have the same data constraint. For these experiments, the unique training data budget D_C is fixed at either 100M, 400M or 1.5B tokens. For each data budget, we train a set of language models with increasing amounts of compute that is allocated to either more parameters or more epochs on the unique training data.

Figure 3 (left) shows the main results for scaling with 100M unique tokens² (see Appendix C for 400M and 1.5B tokens). For 100M tokens, the corresponding one-epoch compute-optimal model

²Although small, for example, this is the order of magnitude of a realistic data constraint reflecting data available after filtering the OSCAR dataset [84] for Basque, Punjabi, or Slovenian.



FLOP budget (C)	Parameters (N)	Training tokens (D)	Data budget (D_C)
9.3×10^{20}	2.8B	55B	$\{55, 28, 18, 14, 11, 9, 4, 1.25\}B$
2.1×10^{21}	4.2B	84B	$\{84, 42, 28, 21, 17, 12, 6, 1.9\}B$
9.3×10^{21}	8.7B	178B	$\{178, 88, 58, 44, 35, 25, 13, 4\}B$

Figure 4: **Validation Loss for Different Data Constraints (IsoFLOP).** Each curve represents the same number of FLOPs spent on an equal-sized model. Colors represent different numbers of epochs due to repeating because of data constraints. Parameters and training tokens are set to match the single-epoch compute-optimal configurations for the given FLOPs. Models trained on data that is repeated for multiple epochs have consistently worse loss and diverge if too many epochs are used. Only loss curves for 8.7B runs are smoothed with an exponential moving average and weight of 0.85.

according to scaling laws from [42] has U_N of approximately 7M parameters (see Appendix B for the scaling coefficients we use). Results show that more than a 50% reduction in loss can be attained by training for several epochs ($R_D > 0$) and increasing model size beyond what would be compute-optimal for 100M tokens ($R_N > 0$). We find the best loss to be at around $20\text{-}60\times$ more parameters and epochs, which corresponds to spending around $7000\times$ more FLOPs. These results suggest that one-epoch models significantly under-utilize their training data and more signal can be extracted by repeating data and adding parameters at the cost of sub-optimal compute utilization.

Figure 3 (right) shows the predicted contours created by fitting our data-constrained scaling laws on 182 training runs. In the single-epoch case ($R_D = 0$) with near compute-optimal parameters ($R_N = 0$) our scaling equation (§3.1) reduces to the Chinchilla equation. In this case, both formulas predict the optimal allocation of compute to parameters and data to be the same, resulting in overlapping efficient frontiers. As data is repeated for more than a single epoch, our fit predicts that excess parameters decay faster in value than repeated data ($R_N^* < R_D^*$). As a result, the data-constrained efficient frontier suggests allocating most additional compute to more epochs rather than more parameters. This contrasts the Chinchilla scaling laws [42], which suggest equally scaling both. However, note that they do not repeat the entire training data and their parametric fit explicitly relies on the assumption that models are trained for a single epoch only. Thus, there is no guarantee that their scaling predictions hold for repeated data.

For all three data budgets, our results suggest that *Allocation* is optimized by scaling epochs faster than parameters. We confirm this at scale by training the data-constrained compute-optimal model for 9.3×10^{21} FLOPs and 25 billion unique tokens as suggested by our efficient frontier. Despite having 27% fewer parameters, this model achieves better loss and downstream performance than the model suggested by the Chinchilla scaling laws (Figure 1 (right) and Table 8). Similarly, the 120 billion parameter Galactica model trained on repeated data should have been significantly smaller according to data-constrained scaling laws (Appendix G). An additional benefit of using a smaller model is cheaper inference, though adding parameters can make it easier to parallelize training across GPUs.